

PR Documentation Auditor - DocAI

Auditoria Automatizada de Documentação em Pull Requests

Equipe 3 Guilherme Pereira · Alexandre Moreno · Stela Nascimento · Reilson Fonseca

Prof. Vinicius Cardoso Garcia · CIn / UFPE

EXPOSIÇÃO

Quem usa o DocAI?

Duas personas centrais guiam as decisões de produto.



Carolina Oliveira

PMO Sênior

Dores

- Depende de devs para entender o impacto
- Processo manual demora 15–20 min por PR
- Risco de mudanças críticas passarem despercebidas

Como o TASI ajuda

- Automatiza a auditoria documental
- Relatório Markdown sem jargão técnico
- Detecta gaps de segurança/compliance

Rafael Santos

Desenvolvedor Full Stack

Dores

- Esquece de atualizar docs após mudanças
- Interrompido para explicar impacto ao PMO
- Falta de clareza sobre quais docs alterar

Como o TASI ajuda

- Informa automaticamente o que documentar
- Reduz retrabalho e revisões manuais
- Agiliza aprovação do PR

Por que o DocAI existe?

O problema é o drift documental: o código muda, a documentação não acompanha.

Pull Request altera API, regra de negócio, infraestrutura ou segurança

Documentação fica defasada e o PMO depende de validação técnica manual

Risco: release com requisito incompleto, gap de compliance ou segurança

O Problema: Drift Documental

A cada Pull Request...

Funcionalidades, APIs, regras de negócio, **infraestrutura** e **segurança** mudam — mas a documentação não acompanha.

O PMO, responsável por releases e auditorias, frequentemente não tem expertise técnica para avaliar sozinho se uma mudança exige atualização documental.

15–20 min

por análise manual

~1 seg

meta com IA ($\leq 30s$)

$\geq 90\%$

precisão na detecção

COMPOSIÇÃO

Aplicação completa

CLI em Node.js/TypeScript que analisa PRs, compara documentação e gera relatório Markdown.

Entrada

- Dados do Pull Request
- Diff + metadados
- Documentação existente

Análise IA

- Extrai mudanças
- Detecta gaps
- Classifica criticidade

Persistência

- Histórico MongoDB
- Rastreabilidade
- Base para feedback learning

Saída
Relatório
Markdown
claro para PMO,
Dev e
Segurança

4 subcomandos: audit • fetch • fetch-and-audit • history

Fora do MVP: dashboard web, autenticação, correção automática de docs, Slack/e-mail e substituição da revisão humana.

Como o DocAI roda no dia a dia

O uso esperado é dentro do fluxo de PR, com execução local ou futura integração via GitHub Actions.

1
**Dev abre ou
atualiza PR**

2
**CLI busca diff e
docs**

3
LLM analisa gaps

4
Relatório é gerado

```
$ npm run build && npm test  
$ tasi fetch-and-audit --owner org --repo app --pr 42 --docs ./docs
```

Resultado: Status • Criticidade • Mudanças identificadas • Gaps • Justificativa • Recomendação

O Que Entregamos: CLI de Auditoria Documental

1

Extração

Diff do PR via GitHub REST
API

2

Análise Semântica

LLM detecta mudanças
relevantes

3

Detecção de Gaps

Comparação com docs
existentes

4

Classificação

4 níveis de criticidade

5

Relatório

Markdown + histórico
MongoDB

Stack: Node.js 20 · TypeScript 5 · Commander.js · Vitest · Mongoose · GitHub Actions · Groq · Gemini

C4 Model — Três Níveis de Visão

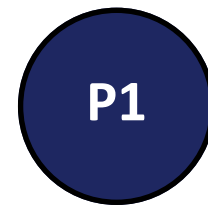


A integração com IA está concentrada no componente LLMIntegration (ADR-005).

Decisões Arquiteturais (ADRs)

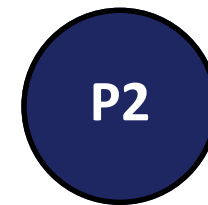
ADR	Decisão	Trade-off principal
ADR-001	GITHUB_TOKEN do Actions	Escopo automático, menos granular que GitHub App
ADR-002	Node.js + TypeScript	Startup rápido, menos libs de LLM que Python
ADR-003	MongoDB para histórico	Flexibilidade JSON, sem transações ACID
ADR-004	Groq-first + fallback Gemini ★	Custo baixo no happy path, segurança nos casos críticos
ADR-005	CLI com Commander.js	Subcomandos profissionais com help/validação
ADR-006	Promoção do extrator de PR	Serviço reutilizável e testável

Prompts Centrais — v2 com Structured Output



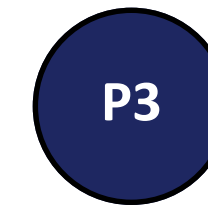
Extração de Mudanças

Identifica mudanças com impacto documental no diff



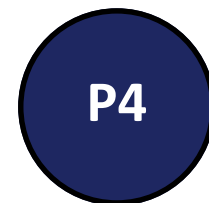
Detecção de Gaps

Compara semanticamente código vs documentação existente



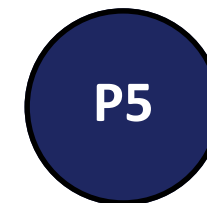
Classificação de Criticidade

4 níveis: Info / Baixa / Alta / Crítica



Geração de Relatório

Markdown estruturado legível pelo PMO sem suporte técnico



Alerta de Mudanças Sensíveis

Dispara roteamento para Gemini em auth / .env / CI/CD

v1 → v2: inclusão de structured output JSON e regras mais rígidas de segurança em todos os prompts

Workflow Document

Economicidade do Desenvolvimento Assistido por IA

R\$ 35,68

Custo real de tokens

421K entrada · 261K saída

16,25h

Esforço humano com IA

4 membros · 4 fases

R\$ 5.762

Custo contrafactual

88h sem IA (estimativa)

Razão de Economicidade

4,98×

Saving estimado

R\$ 4.604 · 79,9%

Análise de Segurança — 10 Achados (AS-01 a AS-10)

P0 — Ações Críticas

AS-01 · Redaction de secrets antes do envio à LLM - Ausência de mascaramento antes do envio para LLM

AS-02 · Falhar o job para criticidade Crítica

AS-03 · MongoDB obrigatório em CI/CD - Persistencia MongoDB opcional conflita com requisito de auditoria

AS-04 · Histórico append-only (imutabilidade)

P1/P2 — Ações Importantes

AS-05 · Resumo sanitizado no comentário do PR

AS-06 · Delimitar dados não confiáveis (prompt injection)

AS-07 · Fail-closed em erro de parsing - Validacao da resposta da LLM usa fallback seguro apenas parcialmente

AS-08 · Permissões mínimas no GITHUB_TOKEN - Permissoes do GitHub Actions precisam ser explicitamente minimas

AS-09 · npm audit (supply chain) - Dependencias npm e supply chain precisam entrar no gate de seguranca

AS-10 · Política de retenção de artefatos (LGPD) - Retencao de artefatos brutos deve ser minimizada

Validação — O Que Funcionou e O Que Falta

✓ Hipótese Confirmada Parcialmente

- ✓ 77 testes passando · 12 arquivos · build limpo
- ✓ ~90% precisão nos cenários simples (mocks)
- ✓ Relatório Markdown compreensível ao PMO
- ✓ Roteamento Groq → Gemini funcional
- ✓ Custo operacional ≈ R\$ 0 (free tier)
- ✓ Fluxo end-to-end com dados simulados

⚠ Itens Pendentes

- Validação end-to-end com APIs reais
- Redaction de secrets (AS-01)
- GitHub Actions workflow em CI/CD
- Gate de supply chain (npm audit)
- Latência real medida (meta: ≤ 30s)
- Feedback learning via histórico MongoDB

Próximos Passos — Três Eixos de Evolução

Técnico	Produto	Adoção
→ GitHub Actions workflow → Redaction de secrets → Fragmentação por arquivo → Fallback p/ modelos maiores	→ Dashboard web → Notificações (Slack/e-mail) → Suporte Swagger / Confluence → Feedback learning (MongoDB)	→ GitHub Marketplace → Suporte multi-linguagem → Modo SaaS → Autenticação de usuários

Decisão estratégica: Perseverar. MVP demonstra viabilidade — próximas iterações focam em validação end-to-end e redaction.

Reflexão Crítica e Uso de IA

01

Meta-prompts detalhados

São a maior alavanca de produtividade com IA generativa.

02

IA não valida build

Números e builds gerados pela IA precisam de verificação humana real.

03

spec → plano → TDD

Este fluxo reduziu retrabalho vs. geração direta de código.

04

Composição foi o mais valioso

C4 Model e ADRs criaram arquitetura que sobreviveu à implementação.

Obrigado!